Banking System Project Analysis

1. Project Overview

This project is a robust, production-grade RESTful API for a Banking System, built using **Node.js, Express, and MongoDB**. It mimics real-world banking operations, including account management, secure authentication, and complex financial transactions using a **double-entry ledger system**.

The project is structured following the **MVC (Model-View-Controller)** architectural pattern, separating data logic, routing, and request handling. It places a strong emphasis on security, validation, observability, and ACID-compliant data operations.

---

2. Architecture & Design Patterns

2.1 Double-Entry Bookkeeping & ACID Transactions

Unlike simple apps that just update a balance field, this system uses a **Double-Entry Ledger**. Every transaction results in at least two ledger entries (a debit from one account and a credit to another), ensuring that money is never created or destroyed out of nowhere.

- It uses **MongoDB Sessions and Transactions** to ensure operations are atomic (either all steps of a transfer succeed, or none do).

2.2 Security & Authentication

- **JWT (JSON Web Tokens)**: Used for stateless authentication.
- **Token Blacklisting**: When a user logs out, their token is saved to a blacklist collection to prevent reuse.
- **Role-Based Access Control (RBAC)**: Distinct roles (e.g., Customer, Admin, System) restrict access to certain endpoints.
- **Rate Limiting**: Defends against brute-force attacks and API abuse.
- **Data Validation**: Uses **Zod** to validate incoming HTTP payloads, ensuring strict typing and schema constraints before data hits the controllers.

2.3 Observability & Error Handling

- **Centralized Error Handling**: A unified error-handling middleware intercepts all errors and formats them into consistent API responses.
- **Logging**: Integrates **Winston** and **Morgan** for structured logging of requests and system events.

---

3. Detailed Component Breakdown

3.1 Data Models (src/models/)

The persistence layer defined using Mongoose schemas:

- **user.model.js**: Stores user information, credentials, and roles. Includes pre-save hooks to hash passwords using bcryptjs.

- **account.model.js**: Represents bank accounts linked to a user. May include account numbers, types (savings/checking), and current status (active/frozen).
- **transaction.models.js**: Records the high-level intent of a money movement (e.g., P2P transfer, deposit, withdrawal) and its status (pending, completed, failed).
- **ledger.model.js**: The core of the accounting engine. Records immutable debit and credit lines linked to transactions. Account balances are dynamically aggregated from this ledger.
- **blacklist.model.js**: Stores invalidated JWT tokens to enforce secure logouts.

3.2 Controllers (src/controllers/)

The business logic layer that handles incoming HTTP requests and returns responses:

- **auth.controller.js**: Handles user registration, login, and secure logout mechanisms.
- **account.controller.js**: Manages the creation of bank accounts, retrieving account details, and viewing aggregated balances.
- **transaction.controller.js**: The most complex controller. Handles deposits, withdrawals, and peer-to-peer (P2P) transfers. Orchestrates MongoDB transactions to ensure the ledger remains balanced.
- **admin.controller.js**: Provides endpoints strictly for system administrators, such as freezing accounts, monitoring system health, or reviewing global transactions.

3.3 Routing (src/routes/)

Maps HTTP endpoints to their respective controller functions:

- **auth.routes.js**: /api/auth/register, /api/auth/login, /api/auth/logout
- **account.routes.js**: /api/accounts/ (creation, fetching user's accounts)
- **transaction.routes.js**: /api/transactions/transfer, /api/transactions/history
- **admin.routes.js**: /api/admin/... protected routes for administrative tasks.

3.4 Middleware (src/middleware/)

Interceptors that process requests before they reach the controllers:

- **auth.middleware.js**: Extracts and verifies the JWT token from request headers or cookies. Checks against the blacklist.
- **authorize.js**: Checks if the authenticated user's role matches the required roles for a route.
- **validate.js**: Intercepts requests and validates req.body, req.query, and req.params against Zod schemas.
- **rateLimiter.js**: Implements express-rate-limit to prevent excessive requests.
- **requestLogger.js**: Middleware to log HTTP request metadata.
- **errorHandler.js**: A global error sink that catches both operational (e.g., insufficient funds) and programming errors, returning a clean JSON response.

3.5 Services & Configuration

- **src/services/email.js**: Uses nodemailer to send transactional emails (e.g., welcome emails, transfer receipts).
- **src/config/db.js**: Manages the connection pool and initialization for the MongoDB cluster.

3.6 Utilities & Validators

- **src/utils/**:

    - AppError.js: A custom error class to differentiate known operational errors from unhandled exceptions.
    - catchAsync.js: A wrapper for async controller functions to eliminate repetitive try-catch blocks and pass errors directly to the errorHandler.
    - logger.js: Winston logger configuration (transports, formatting).

- **src/validators/**:
    - auth.validator.js, account.validator.js, transaction.validator.js: Zod schema definitions corresponding to specific endpoints.

---

4. System Architecture Diagram (Text Representation)

```
[ Client (Web/Mobile App) ]
          |
          v
[ HTTP Request (Express.js) ]
          |
          v
[ Rate Limiter Middleware ] ---> (Rejects if too many requests)
          |
          v
[ Request Logger (Morgan/Winston) ]
          |
          v
[ Auth Middleware ] ---> (Validates JWT & Checks Blacklist)
          |
          v
[ Role Authorization ] ---> (Ensures Admin/Customer access)
          |
          v
[ Zod Validation ] ---> (Validates body/params. Rejects if invalid)
          |
          v
[ Controller Layer ] ---> [ Services (e.g., Email Notifications) ]
          |
          v
[ MongoDB Transactions (ACID) ]
          |
  +-----------------+-----------------+
  |                 |                 |
[Users]         [Accounts]      [Transactions & Ledger]
```

Conclusion

This Banking System is engineered with enterprise-level concepts. By employing strict input validation, comprehensive logging, stateless JWT auth with a revocation mechanism, and a double-entry ledger backed by MongoDB ACID transactions, the project effectively models the rigorous constraints required in real-world financial technology applications.